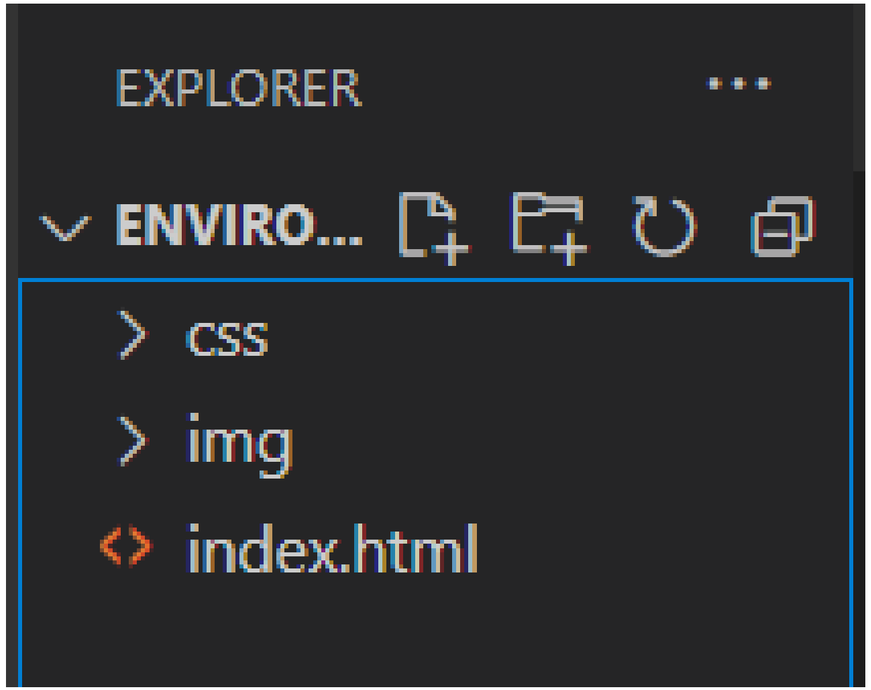
Intégration

HTML & CSS

Le HTML et les différents éléments de base à connaitre par coeur

1 - L’environnement de travail



2 - Structure d’une page web

index.html > html > head > title

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
</html>
```

3 - Les balises structurantes

```html
<body>
    <header>
        <!-- entête de la page -->
    </header>

    <main>
        <!-- contenu de la page -->
        <section>
            <!-- Partie de contenu de la page -->
        </section>
    </main>

    <footer>
        <!-- Pied de page -->
    </footer>
</body>
```

4 - Les règles a connaitre

1. Le logo est cliquable et ramène obligatoirement la page d’accueil
2. Le menu de navigation doit toujours se trouver dans une balise nav
3. Le menu de navigation est une liste de lien

5 - La structure du header

```html
<header>
    <!-- Le logo est cliquable et ramène à la page d'accueil -->
    <a href="/">
        <!-- La balise image doit toujours comporter un texte alternatif -->
        <img src="img/logo.png" alt="Logo">
    </a>

    <nav>
        <!-- La balise nav indique la présence d'un menu de navigation -->
        <ul>
            <!-- Le menu est toujours une liste de lien. On utilise donc une liste ordonnée ul -->
            <li><a href="#">Accueil</a></li>
            <li><a href="#">Blog</a></li>
            <li><a href="#">Contact</a></li>
        </ul>
    </nav>
</header>
```

6 - Les balises à connaitre par coeur

Il existe différents type de balises. Certaines balises s’ouvrent et se ferment, ce sont les balises qui peuvent inclure d’autres éléments a l’intérieur. D’autres balises, ne comportant aucun élément à l’intérieur possède uniquement qu’une balise d’ouverture.

6 - Les balises à connaitre par coeur

Par exemple, les balises de titre, de lien ou de paragraphes possèdent des éléments a l’intérieur. Ce sont des balises ouvrantes et fermantes.

La balise image ne comporte aucun élément. Elle s’ouvre mais ne se ferme pas.

6 - Les balises à connaitre par coeur

Balises de structures :

- **Header** : Balise d’entête
- **Nav** : Indique la présence d’une navigation
- **Main** : Contenu principal de la page
- **Section** : Sectionne les différentes partie d’une page
- **Aside** : Permet d’ajouter du contenu complémentaire
- **Footer** : Pied de page
- **Article** : Créer un bloc d’information logique

6 - Les balises à connaitre par coeur

Balises de structures :

- **h1,h2,...,h6** : Titres du niveau 1 au niveau 6
- **a** : Lien
- **p** : Paragraphe
- **ul** : Liste à puce
- **ol** : Liste ordonnée
- **li** : Element de liste à puce ou ordonnée
- **img** : Balise d’image

7 - La liaison avec fontawesome

Font awesome est une bibliothèque de font d'icone en ligne qui propose de nombreuses icones gratuitement. Il faut suivre les étapes sur le site pour obtenir un lien qu'on insèrera dans la balise head.

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://kit.fontawesome.com/66dfd999d9.js" crossorigin="anonymous"></script>
    <link rel="stylesheet" href="css/style.css">
    <title>Flex and position project</title>
```

8 - Normaliser le css

Afin de se faciliter la tâche lors de l’intégration d’une maquette. Il faut :

- Identifier les éléments de design sur la maquette (taille de texte, éléments visuels qui se répètent, etc.
- Repérer les différentes couleurs.
- Identifier les différentes intéractions

La préparation préalable prend un peu de temps mais permet d’être plus efficace et de gagner du temps à terme.

8 - Normaliser le css

Lorsque les éléments sont identifié, il faut normaliser votre document comme dans l’exemple ci-dessous.

```
/* General */
*{
    margin:0;
    scroll-behavior: smooth;
    font-family: var(--open);
}

h1,
h2{
    font-size: 22px;
    font-weight: 700;
    color:var(--black);
}
```

```
    font-weight: 400;
    color:var(--black);
}

a{
    font-size: 16px;
    font-weight: 400;
    text-decoration: none;
    transition:all 0.2s ease-in-out;
    color:var(--black);
}

a:hover{
    transition:all 0.2s ease-in-out;
    color:var(--red);
}

.container-width-1440{
    max-width: 1440px;
    margin:auto;
    padding:0 15px;
}
```

9 - Utiliser des fonts google

Afin de pouvoir utiliser les fonts google, vous avez 2 possibilités après avoir séléctionner vos fonts.

Intégré vos fonts via link sur le html

ou

Intégré vos fonts sur le css

9 - Utiliser des fonts google

Sur le site, vous pouvez récupérer le code et l’intégré comme suit dans le css. N’oubliez pas de récupérer également le font-family afin de pouvoir utilisé la font sur les éléments choisis.

```
/* Fonts */
@import url('https://fonts.googleapis.com/css2?family=Open+Sans:ital,wght@0,300..800;1,300..800&display=swap');
```

10 - Utiliser des variables

Lors de la mise en place de vos projets, certaines informations peuvent se répéter et même changer durant la vie de votre projet, comme par exemple, les couleurs.

Voici comment créer et utiliser les variables :

```
:root{
    --primary: #000;
    --secondary: #f60303;
}

p{
    color:var(--secondary);
}
```

11 - Les selecteurs en CSS

Créer des éléments en HTML c’est une très bonne étape cependant, il va falloir apprendre à les sélectionner afin de pouvoir leur appliquer du style.
Il existe une multitude de sélecteurs. Nous allons voir ici les plus basiques et courants.

- Le nom de balise : Exemple p{} pour un paragraphe
- La classe : exemple : .nomdelaclasse{}
- L’ID : exemple : #identifiant{}

11 - Les selecteurs en CSS

Lorsqu’on crée un élément et qu’on utilise les sélecteurs nous pouvons utiliser des chemins directs ou indirectes ce qui va influencer le comportement du CSS.

```
header a{
    /* Ce sélecteur séléctione tous les liens présent dans le header */
}

header > a{
    /* Ce sélecteur séléctionne les élements a en lien direct avec le header */
}
```

11 - Les selecteurs en CSS

Nous pouvons également choisir d’ajouter des modifications de l’état lors du survol par exemple.

```
header a{
    /* Ici il faudra définir l'état de base de l'élement */
}

header a:hover{
    /* Ici nous modifirons sont état, par exemple le chagement d'une couleur
    lorsque nous passons la souris dessus l'élément concerné */
}
```

12 - Les propriétés de base du CSS

Il existe une multitude de propriétés qui sont applicables, pour cela il faut bien comprendre la logique utilisé.

En CSS, l’anglais est de rigueur, il faut donc penser de manière logique en anglais.

Par exemple, si nous souhaitons appliquer une taille de texte à un paragraphe, on utilisera la propriété font-size. Font faisant référence à la typographie et size à la taille.

De même pour la graisse, on utilisera font-weight. Font pour identifier que nous parlons d’une typographie et weight pour parler de la graisse.

12 - Les propriétés de base du CSS

Si nous continuons sur cette logique, tous les éléments sont concernés.

Pour les texte et les font la propriété commencera par **font** ou pas **text**.

Pour les espaces on utilisera **margin** pour les espace externes et **padding** pour les espaces internes.

Pour les bordures on utilisera le terme **border**.

12 - Les propriétés de base du CSS

Pour les arrière plan, il faudra utiliser les propriétés qui commencent par **background**.

Pour les listes, il faut utiliser les propriétés qui commencent par **list**.

En ce qui concerne les positions et les tailles, cela reste encore une fois logique. **Width** pour taille, **height** pour la hauteur etc.

L’important est de savoir qu’il existe une propriété permettant de réaliser ce que l’on souhaite.

Apprendre par coeur n’est pas la meilleure solution.

12 - Les propriétés de base du CSS

Les mauvaises pratique à ne pas faire :

- Apprendre par coeur sans comprendre
- Attendre que la solution viennent d’elle-même
- Compter sur un collegue, un camarade ou un professeur

Les bonnes pratiques :

- Déterminer ce dont on à besoin
- Effectuer une recherche
- Tester vos solutions
- Utiliser les outils (memo, tutos, chat gpt, etc.)
- Essayer de comprendre la solution que vous avez mise en place

Flexbox

Flexbox

Display : Flex

Flexbox

Flex Direction

La propriété "flex-direction" contrôle l'axe principal le long duquel les flex-items sont disposés à l'intérieur d'un conteneur flex. Voici les différentes valeurs que vous pouvez utiliser :

- **row** : Les flex-items sont disposés horizontalement de gauche à droite (le sens de l'écriture dans les langues occidentales).
- **row-reverse** : Les flex-items sont disposés horizontalement de droite à gauche.
- **column** : Les flex-items sont disposés verticalement de haut en bas.
- **column-reverse** : Les flex-items sont disposés verticalement de bas en haut.

Flexbox

Flex Wrap

La propriété "flex-wrap" détermine si les flex-items peuvent être enroulés sur plusieurs lignes (ou colonnes dans le cas d'une disposition verticale). Voici les valeurs que vous pouvez utiliser :

- **nowrap** : Les flex-items sont disposés sur une seule ligne (ou colonne) et ne sont pas enroulés.
- **wrap** : Les flex-items peuvent être enroulés sur plusieurs lignes (ou colonnes) si nécessaire pour s'adapter à l'espace disponible.
- **wrap-reverse** : Les flex-items sont enroulés sur plusieurs lignes (ou colonnes), mais le sens de l'enroulement est inversé par rapport à la direction principale.

Flexbox

Flex flow

La propriété "flex-flow" est un raccourci pratique qui permet de définir à la fois la direction principale (flex-direction) et le comportement de l'enroulement (flex-wrap) des flex-items en une seule déclaration.

Par exemple, en utilisant "flex-flow: column wrap", vous spécifiez que les flex-items seront disposés en colonnes de haut en bas et qu'ils pourront être enroulés sur plusieurs colonnes si nécessaire pour s'adapter à l'espace disponible.

Cette syntaxe concise simplifie la gestion de la mise en page flexbox en regroupant deux propriétés connexes en une seule, ce qui permet un code plus clair et plus facile à lire.

Flexbox

Justify content

La propriété "justify-content" contrôle l'alignement des flex-items le long de l'axe principal du conteneur flex. Voici les valeurs que vous pouvez utiliser :

- **flex-start** : Les flex-items sont alignés au début de l'axe principal.
- **flex-end** : Les flex-items sont alignés à la fin de l'axe principal.
- **center** : Les flex-items sont centrés le long de l'axe principal.
- **space-between** : Les flex-items sont répartis également le long de l'axe principal, avec un espace vide entre eux.
- **space-around** : Les flex-items sont répartis également le long de l'axe principal, avec un espace vide autour d'eux.
- **space-evenly** : Les flex-items sont répartis également le long de l'axe principal, avec un espace vide équivalent entre eux.
- **start** et **end** : Alternatives à flex-start et flex-end, basées sur la direction de l'écriture.
- **left** et **right** : Alternatives à flex-start et flex-end pour une direction horizontale.
- **safe** et **unsafe** : Variantes de flex-start et flex-end prenant en compte la direction d'écriture sécurisée et non sécurisée, respectivement.

Flexbox

Flex item

La propriété "align-items" contrôle l'alignement des flex-items le long de l'axe transversal du conteneur flex. Voici les valeurs que vous pouvez utiliser :

- **stretch** : Les flex-items sont étirés pour occuper toute la hauteur (ou largeur) du conteneur flex.
- **flex-start** : Les flex-items sont alignés au début de l'axe transversal.
- **flex-end** : Les flex-items sont alignés à la fin de l'axe transversal.
- **center** : Les flex-items sont centrés le long de l'axe transversal.
- **baseline** : Les flex-items sont alignés en fonction de leur ligne de base.

Flexbox

Align Content

La propriété "align-content" contrôle l'alignement des lignes flexibles à l'intérieur du conteneur flex lorsque l'espace disponible sur l'axe transversal est plus grand que nécessaire pour les disposer. Voici les valeurs que vous pouvez utiliser :

- **stretch** : Les lignes flexibles sont étirées pour occuper toute la hauteur (ou largeur) du conteneur flex.
- **flex-start** : Les lignes flexibles sont alignées au début de l'axe transversal.
- **flex-end** : Les lignes flexibles sont alignées à la fin de l'axe transversal.
- **center** : Les lignes flexibles sont centrées le long de l'axe transversal.
- **space-between** : Les lignes flexibles sont réparties également sur tout l'axe transversal, avec un espace vide entre elles.
- **space-around** : Les lignes flexibles sont réparties également sur tout l'axe transversal, avec un espace vide autour d'elles.

Flexbox

Gap, Row Gap, Column Gap

Les propriétés "gap", "row-gap" et "column-gap" permettent de définir des espaces entre les flex-items à l'intérieur du conteneur flex. Voici comment elles fonctionnent :

- **gap** : La propriété "gap" définit l'espace entre les flex-items sur les deux axes, à la fois sur l'axe principal et l'axe transversal.
- **row-gap** : La propriété "row-gap" définit l'espace vertical entre les lignes flexibles, c'est-à-dire l'espace entre les flex-items lorsqu'ils sont disposés sur des lignes flexibles supplémentaires en raison du "wrap" ou du "wrap-reverse".
- **column-gap** : La propriété "column-gap" définit l'espace horizontal entre les flex-items lorsqu'ils sont disposés en colonnes.

Flexbox

Order

La propriété "order" permet de spécifier l'ordre dans lequel les flex-items sont affichés à l'intérieur du conteneur flex. Par défaut, tous les flex-items ont un ordre de 0.

En attribuant des valeurs numériques à la propriété "order", vous pouvez modifier l'ordre de présentation des flex-items. Les flex-items avec des valeurs "order" inférieures seront affichés en premier, tandis que ceux avec des valeurs "order" supérieures seront affichés plus tard.

Flexbox

Flex Grow, Flex Shrink, Flex Basis

Les propriétés "flex-grow", "flex-shrink" et "flex-basis" sont utilisées ensemble pour déterminer la taille et le comportement de répartition de l'espace des flex-items à l'intérieur du conteneur flex.

- **flex-grow** : Détermine la capacité d'un flex-item à s'étirer pour remplir l'espace disponible dans le conteneur flex. Plus la valeur est élevée, plus l'élément s'étire.
- **flex-shrink** : Détermine la capacité d'un flex-item à se réduire pour s'adapter à l'espace disponible dans le conteneur flex. Plus la valeur est élevée, plus l'élément se rétrécit.
- **flex-basis** : Définit la taille de base initiale d'un flex-item avant la distribution de l'espace restant. Cela peut être une valeur fixe comme "200px" ou une proportion de l'espace disponible comme "50%".

Flexbox

Flex

La propriété raccourcie "flex" permet de définir simultanément les valeurs de "flex-grow", "flex-shrink" et "flex-basis" en une seule déclaration. Voici la syntaxe :

flex: [flex-grow] [flex-shrink] [flex-basis];

- **flex-grow** : Détermine la capacité d'un flex-item à s'étirer pour remplir l'espace disponible.
- **flex-shrink** : Détermine la capacité d'un flex-item à se réduire pour s'adapter à l'espace disponible.
- **flex-basis** : Définit la taille de base initiale du flex-item avant la distribution de l'espace restant.

Par exemple, en utilisant "flex: 1 0 auto;", vous spécifiez que le flex-item peut s'étirer pour remplir l'espace disponible, mais ne doit pas se rétrécir, et sa taille de base est automatiquement calculée en fonction de son contenu.

Flexbox

Align Self

La propriété "align-self" permet de remplacer ou de compléter la valeur "align-items" définie sur le conteneur flex pour un flex-item spécifique. Elle contrôle l'alignement d'un flex-item sur l'axe transversal du conteneur flex. Voici les valeurs que vous pouvez utiliser :

- **stretch** : Étire le flex-item pour occuper toute la hauteur (ou largeur) du conteneur flex.
- **flex-start** : Aligne le flex-item au début de l'axe transversal.
- **flex-end** : Aligne le flex-item à la fin de l'axe transversal.
- **center** : Centre le flex-item le long de l'axe transversal.
- **baseline** : Aligne le flex-item en fonction de sa ligne de base par rapport aux autres flex-items.

IMPORTANT

Il est absolument inutile d’apprendre par coeur tous les éléments que nous avons vu précedemment. L’important est de savoir chercher et utiliser ce dont vous avez besoin au moment ou vous en avez besoin.

Autre informations

Il existe de nombreuses façon de mettre en forme un contenu. Les flexbox sont très utiles mais ne sont pas obligatoire dans tous les cas de figure. Les autres display sont également très importans.

Je vous invite à également prendre connaissance du display grid qui dans certains cas est plus adapté.

Autre informations

Pour le projet fourni il vous faudra rechercher d’autres éléments à savoir :

- **Position relative et absolute**
- **Z-index**
- **::before et ::after**